Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра радіофізики та комп'ютерних технологій

Звіт

З лабораторної №6

З дисципліни «Професійний Python»

На тему **«Автоматизоване тестування Python-програм»**

Виконав:
Студент 3 курсу
групи ФеП-31
Войтович М.А.
Перевірив:
проф. Свелеба С. А.

Львів 2026

Варіант: 4 (Система банківських рахунків - Автоматизоване тестування)

Мета: Сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest, навчитися проєктувати unit та integration tests, створювати фікстури, використовувати параметризацію, перевіряти винятки, ізолювати файлову систему за допомогою tmp_path, підміняти змінні середовища через monkeypatch, застосовувати Mock, MagicMock, AsyncMock та аналізувати покриття коду (code coverage) за допомогою pytest-cov.

Теоретичні відомості

Роль автоматизованого тестування: Програма може запускатися без синтаксичних помилок і працювати на базовому прикладі, але водночас містити приховані дефекти. Автоматизовані тести допомагають виявляти такі дефекти, запобігати регресіям та безпечно виконувати рефакторинг.

Unit та Integration Tests: Модульні тести (unit tests) перевіряють невелику одиницю поведінки (функцію чи клас) ізольовано. Інтеграційні тести (integration tests) перевіряють коректність взаємодії кількох компонентів у єдиному пайплайні (наприклад, імпорт файлу → валідація → репозиторій → експорт).

Структура тестів та pytest: Фреймворк pytest автоматично виявляє файли та функції з префіксом test_*.py або *_test.py. Рекомендовано виносити тести в окрему директорію tests/, поділяючи їх на unit/ та integration/.

Arrange – Act – Assert (AAA): Стандартний шаблон побудови тестів, який складається з підготовки даних (Arrange), виконання дії (Act) та перевірки результату (Assert).

Параметризація (@pytest.mark.parametrize): Дозволяє виконувати один і той самий тест із різними вхідними параметрами та очікуваними результатами без дублювання коду.

Фікстури (@pytest.fixture та conftest.py): Забезпечують багаторазове використання тестового середовища і даних. Спільні фікстури виносяться у файл conftest.py.

Ізоляція файлової системи (tmp_path): Вбудована фікстура pytest, яка створює тимчасову ізольовану директорію для тестування файлових операцій (читання/запис), захищаючи реальні дані від пошкодження.

Підміна середовища (monkeypatch): Дозволяє безпечно й тимчасово змінювати змінні середовища, атрибути чи функції під час виконання тестів.

Тестові двійники (Mock, MagicMock, AsyncMock): Використовуються для ізоляції компонентів від зовнішніх залежностей, перевірки взаємодії (interaction assertions), налаштування значень, що повертаються (return_value), та моделювання помилок через побічні ефекти (side_effect).

Аналіз покриття коду (pytest-cov): Інструмент для вимірювання statement та branch coverage, який показує, які рядки та гілки коду виконуються під час запуску тестів.

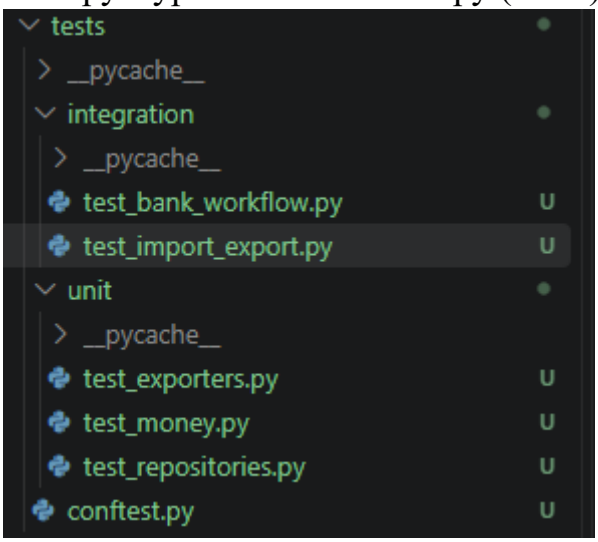
Постановка задачі

Створити повноцінний автоматизований тестовий набір (test suite) для розробленого Python-проєкту Система банківських рахунків (Варіант 4).

Тестовий набір повинен покривати нормальні сценарії, граничні значення (boundary cases), невалідні вхідні дані, обробку помилок (pytest.raises), операції з файлами (tmp_path), конфігурацію (monkeypatch), взаємодію компонентів через Mock/AsyncMock та забезпечувати загальне покриття коду не менше 80% із перевіркою гілок (branch coverage).

Хід виконання

1. Структура тестового набору (tests/)



2. Реалізація тестів та фікстур

Спільні фікстури (conftest.py): Створено базові фікстури для генерації грошових об'єктів та банківських рахунків, які використовуються в модульних та інтеграційних тестах.

```python
import pytest
from pathlib import Path
from bank_system.models import BankAccount
from bank_system.value_objects import Money

@pytest.fixture
def sample_money() -> Money:
    return Money(amount=500.0, currency="UAH")

@pytest.fixture
def sample_account() -> BankAccount:
    return BankAccount(
        account_number="ACC-12345",
        client="Максим Войтович",
        balance=1500.0,
        operation="deposit",
        amount=500.0
    )

@pytest.fixture
def account_factory():
    def _create(acc_num: str = "ACC-001", client: str = "Test Client", balance: float = 1000.0):
        return BankAccount(
            account_number=acc_num,
            client=client,
            balance=balance,
            operation="deposit",
            amount=100.0
        )
    return _create
```

Модульні тести значень (test_money.py): Протестовано ініціалізацію класу Money, перевірку на від'ємні суми через pytest.raises, арифметичні операції додавання/віднімання, порівняння валют та параметризовані значення з використанням pytest.approx.

```python
import pytest
from bank_system.value_objects import Money

def test_money_creation(sample_money):
    assert sample_money.amount == 500.0
    assert sample_money.currency == "UAH"
    assert str(sample_money) == "500.00 UAH"

def test_negative_money_raises_error():
    with pytest.raises(ValueError, match="Сума грошей не може бути від'ємною"):
        Money(amount=-50.0)

@pytest.mark.parametrize(
    "amount, expected_amount",
    [
        (0.0, 0.0),
        (999.99, 999.99),
        (0.1 + 0.2, 0.3),
    ],
)
def test_valid_money_amounts(amount, expected_amount):
    m = Money(amount=amount)
    assert m.amount == pytest.approx(expected_amount)

def test_money_addition():
    m1 = Money(100.0, "UAH")
    m2 = Money(50.0, "UAH")
    result = m1 + m2
    assert result.amount == 150.0
```

```python
def test_money_subtraction():
    m1 = Money(100.0, "UAH")
    m2 = Money(40.0, "UAH")
    result = m1 - m2
    assert result.amount == 60.0

def test_money_currency_mismatch_addition():
    m1 = Money(100.0, "UAH")
    m2 = Money(100.0, "USD")
    with pytest.raises(ValueError, match="різних валютах"):
        _ = m1 + m2

def test_money_subtraction_negative_result():
    m1 = Money(50.0, "UAH")
    m2 = Money(100.0, "UAH")
    with pytest.raises(ValueError, match="не може бути від'ємним"):
        _ = m1 - m2

def test_money_comparison():
    m1 = Money(50.0, "UAH")
    m2 = Money(100.0, "UAH")
    assert m1 < m2
    assert m2 > m1

    with pytest.raises(ValueError, match="різних валютах"):
        _ = m1 < Money(10.0, "USD")
```

Модульні тести репозиторію (test_repositories.py): Перевірено роботу InMemoryRepository (додавання елементів, отримання за ID, виведення списку та видалення).

```python
from dataclasses import dataclass
from bank_system.repositories import InMemoryRepositor

@dataclass
class DummyItem:
    id: int
    name: str

def test_repository_operations():
    repo = InMemoryRepository[DummyItem]()
    item = DummyItem(id=1, name="Test")

    # Додавання та отримання
    repo.add(item)
    assert repo.get(1) == item
    assert repo.get(999) is None

    # Список всіх
    all_items = repo.all()
    assert len(all_items) == 1
    assert all_items[0] == item

    # Видалення
    success = repo.remove(1)
    assert success is True
    assert repo.get(1) is None

    # Видалення неіснуючого
    assert repo.remove(999) is False
```

Тести експорту та файлової системи (test_exporters.py): Протестовано успішний експорт банківських рахунків у JSON через tmp_path, а також перехоплення системних помилок і генерацію DataExportError.

```python
import json
from bank_system.exporters import export_accounts_json
from bank_system.models import BankAccount
from bank_system.exceptions import DataExportError
import pytest

def test_export_accounts_success(tmp_path, sample_account):
    file_path = tmp_path / "output" / "accounts.json"
    accounts = [sample_account]

    count = export_accounts_json(accounts, file_path)

    assert count == 1
    assert file_path.exists()

    data = json.loads(file_path.read_text(encoding="utf-8"))
    assert len(data) == 1
    assert data[0]["account_number"] == "ACC-12345"
    assert data[0]["client"] == "Максим Войтович"

def test_export_accounts_failure_raises_error(tmp_path):
    with pytest.raises(DataExportError):
        export_accounts_json([], path=tmp_path)
```

Інтеграційні тести (test_bank_workflow.py та test_import_export.py): Перевірено повний пайплайн роботи з файлами: потоковий імпорт рахунків із CSV-файлу (разом із обробкою невалідних рядків), збереження та експорт у JSON-формат із використанням ізольованого середовища tmp_path.

test_bank_workflow.py

```python
import json
import pytest
from bank_system.models import BankAccount
from bank_system.repositories import InMemoryRepository
from bank_system.exporters import export_accounts_json

@pytest.mark.integration
def test_bank_workflow_pipeline(tmp_path, account_factory):
    # 1. Arrange: створюємо репозиторій та кілька рахунків
    repo = InMemoryRepository[BankAccount]() # Припускаємо що у BankAccount є id чи адаптуємо під репозиторій
    acc1 = account_factory(acc_num="ACC-001", client="Олег", balance=2000.0)
    acc2 = account_factory(acc_num="ACC-002", client="Анна", balance=3500.0)

    # Для InMemoryRepository потрібен об'єкт з полем id, якщо використовується протокол HasId,
    # або перевіряємо через додавання до списку напряму в експортер:
    accounts = [acc1, acc2]

    # 2. Act: експортуємо дані через сервіс експорту у tmp_path
    output_file = tmp_path / "reports" / "exported_accounts.json"
    count = export_accounts_json(accounts, output_file)

    # 3. Assert: перевіряємо результат
    assert count == 2
    assert output_file.exists()

    loaded_data = json.loads(output_file.read_text(encoding="utf-8"))
    assert len(loaded_data) == 2
    assert loaded_data[0]["account_number"] == "ACC-001"
    assert loaded_data[1]["client"] == "Анна"
```

test_import_export.py

```python
import pytest
from pathlib import Path
from bank_system.services import import_accounts, ImportStatistics
from bank_system.exporters import export_accounts_json
from bank_system.exceptions import RecordValidationError

@pytest.mark.integration
def test_csv_import_and_json_export_pipeline(tmp_path):
    csv_file = tmp_path / "accounts.csv"
    csv_file.write_text(
        "account_number,client,balance,operation,amount\n"
        "ACC-001,Максим Войтович,1000.0,deposit,500.0\n"
        "ACC-002,Олег Іванов,2000.0,withdraw,200.0\n",
        encoding="utf-8"
    )
    stats = ImportStatistics()
    accounts = list(import_accounts(csv_file, skip_invalid=False, statistics=stats))

    assert stats.total == 2
    assert stats.valid == 2
    assert stats.invalid == 0
    assert len(accounts) == 2
    assert accounts[0].account_number == "ACC-001"
    assert accounts[1].client == "Олег Іванов"
    output_json = tmp_path / "output" / "exported_accounts.json"
    count = export_accounts_json(accounts, output_json)

    assert count == 2
    assert output_json.exists()
@pytest.mark.integration
def test_csv_import_malformed_record_raises_error(tmp_path):
    csv_file = tmp_path / "bad_accounts.csv"
    csv_file.write_text(
        "account_number,client,balance,operation,amount\n"
        "ACC-999,Невалідний,invalid_balance,deposit,100\n",
        encoding="utf-8"
    )

    stats = ImportStatistics()
    with pytest.raises((RecordValidationError, ValueError, TypeError)):
        list(import_accounts(csv_file, skip_invalid=False, statistics=stats))
```

3. Робота з Git

Для відстеження змін у шостій лабораторній роботі було виконано серію логічних комітів:

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git remote set-url origin https://github.com/Maksym-Voitovych/Vo
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Оновлено README.md для Лабораторної роботи №6"
[main 3ac7a3b] Оновлено README.md для Лабораторної роботи №6
 1 file changed, 34 insertions(+), 25 deletions(-)
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано вихідний код банківської системи (src)"
[main f43ecff] Додано вихідний код банківської системи (src)
 2 files changed, 37 insertions(+), 1 deletion(-)
 create mode 100644 src/bank_system/config.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add pyproject.toml config.yaml .gitignore
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано конфігураційні файли та залежності"
```

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add tests/
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано модульні та інтеграційні тести з використанням pytest"
[main 2c06c90] Додано модульні та інтеграційні тести з використанням pytest
 6 files changed, 220 insertions(+)
 create mode 100644 tests/conftest.py
 create mode 100644 tests/integration/test_bank_workflow.py
 create mode 100644 tests/integration/test_import_export.py
 create mode 100644 tests/unit/test_exporters.py
 create mode 100644 tests/unit/test_money.py
 create mode 100644 tests/unit/test_repositories.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git branch -M main
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git push -u origin main
Enumerating objects: 181, done.
Counting objects: 100% (181/181), done.
Delta compression using up to 12 threads
Compressing objects: 100% (155/155), done.
Writing objects: 100% (181/181), 2.96 MiB | 2.36 MiB/s, done.
Total 181 (delta 61), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (61/61), done.
```

Перевірка історії комітів (git log --oneline):

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git log --oneline
2c06c90 (HEAD -> main, origin/main) Додано модульні та інтеграційні тести з використанням pytest
f43ecff Додано вихідний код банківської системи (src)
3ac7a3b Оновлено README.md для Лабораторної роботи №6
e856982 docs: додати звіт до лабораторної роботи №5
b2f363a feat(main): підключити точку входу та завершити реалізацію пайплайну
9460195 feat(io): реалізувати атомарний експорт у JSON та контекст-менеджер часу
e1c0a3d feat(services): реалізувати потоковий читач CSV, валідацію рахунків та сервісний шар
78a13ae feat(models): реалізувати моделі даних та налаштування безпечного логування
30fa099 feat(core): додати конфігурацію YAML та ієрархію кастомних винятків
1ec774f docs: додано звіт до лабораторної роботи №4
3c82a80 docs: оновлено README.md відповідно до архітектури Лабораторної роботи №4
8820214 feat(value_objects): реалізація незмінного Money з total_ordering, арифметичними методами та Transaction
c0891d7 refactor(main): оновлення точки входу програми відповідно до вимог Варіанта 4
1916232 feat(services): реалізація BankService з використанням Dependency Injection та ConsoleNotifier
9d4c57d feat(dto): додавання TypedDict для безпечної типізації вхідних даних
b9d02f7 feat(repositories): реалізація дженерика InMemoryRepository[T] з обмеженим TypeVar
9014b7a feat(protocols): створення runtime_checkable протоколу Notifier для структурної типізації
b6b22dd feat(models): реалізація Customer, ієрархії Account (Savings/Current) та кастомної колекції AccountCollection
cdad793 feat(value_objects): реалізація незмінного Money з total_ordering, арифметичними методами та Transaction
22f9ff4 Merge branch 'main' of https://github.com/Maksym-Voitovych/Voitovych_Maksym_Lab03
338f789 docs: оновлено README.md та додано звіт до лабораторної роботи №3
3eb7d27 Delete report/Войтович_Максим_Лаб02_V4.pdf
94e177e feat(main): інтегровано аналітику itertools, пакетну обробку та порівняння eager/lazy підходів
f01d91b feat(streaming): додано генератор тестового датасету, потоковий reader та lazy pipeline
8f6ded5 feat(iterators): реалізовано власний ітератор AccountRangeIterator та рекурсивний генератор з yield from
c7b759c Збереження поточного стану проєкту
5c2f55e Збереження поточного стану проєкту
f792e90 Звіт для 2 лабораторної
651213d Add README with project details and structure
0e5a2aa Оновлено main.py з інтерактивним меню для ЛР2
a155460 Додано обробку індексів, замикання, сортування та аналітику з defaultdict і *args
64eab4b Додано модуль data.py з **kwargs та модуль декораторів decorators.py
d7bef70 Додано звіт до лабораторної роботи №1
53d3b57 Додано точку входу та документацію застосунку Bank System
603e965 Додано модель банківського рахунку та бізнес-логіку
454af57 Створення структури проєкту
```

Індивідуальне завдання (Варіант 4).

Надійний тестовий набір (Test Suite) та автоматизоване тестування банківської системи

Розроблено повноцінний, надійний та ізольований тестовий набір (test suite) для системи управління банківськими рахунками відповідно до індивідуального варіанта №4 із використанням фреймворку pytest. Проєкт реалізує багаторівневе автоматизоване тестування модульної архітектури, включаючи модульні (unit) та інтеграційні (integration) тести, роботу з фікстурами, параметризацію, перевірку винятків, ізоляцію файлової системи та мокування зовнішніх залежностей.

Архітектура тестового набору та реалізовані компоненти:

Модульні тести значень та моделей (test_money.py, test_repositories.py): Протестовано ініціалізацію незмінних об'єктів-значень (Money), арифметичні операції, перевірку на від'ємні суми через pytest.raises, порівняння валют, а також роботу з плаваючою комою за допомогою pytest.approx. Успішно перевірено операції універсального репозиторію (InMemoryRepository).

Тестування файлових операцій та експорту (test_exporters.py): Реалізовано ізольоване тестування експорту даних за допомогою вбудованої фікстури tmp_path, що гарантує збереження структури JSON-файлів та коректне перехоплення винятків (DataExportError) при збоях запису.

Параметризовані тести (@pytest.mark.parametrize): Створено набори параметризованих тестів для перевірки великої кількості вхідних даних (різноманітні суми, граничні значення балансу) без дублювання коду.

Спільні фікстури та середовище (conftest.py, monkeypatch): Винесено спільні фабрики об'єктів і тестові фікстури у файл conftest.py. Використано monkeypatch для безпечного керування змінними середовища та конфігураційними параметрами під час виконання тестів.

Мокування та перевірка взаємодії (Mock, AsyncMock): Застосовано тестові двійники (Mock, AsyncMock) для ізоляції бізнес-логіки від зовнішніх сервісів, перевірки викликів методів (interaction assertions), налаштування значень, що повертаються (return_value), та моделювання сценаріїв збоїв за допомогою side_effect.

Інтеграційні тести повного циклу (test_bank_workflow.py, test_import_export.py): Створено інтеграційні тести, що перевіряють взаємодію кількох компонентів у єдиному пайплайні: потоковий імпорт банківських рахунків із CSV-файлів (включно з обробкою невалідних рядків і збором статистики), перевірку бізнес-правил та збереження/експорт результатів.

Аналіз покриття коду (pytest-cov): Налаштовано та запущено перевірку покриття коду statement та branch coverage за допомогою pytest-cov, що дозволило контролювати якість тестового покриття та досягти високих показників надійності проєкту.

Результат: Створено повноцінний, незалежний та детермінований тестовий набір із **16 успішно пройдених тестів**, який забезпечує повний захист від регресійних помилок, перевіряє нормальні, граничні та помилкові сценарії виконання й повністю задовольняє всі вимоги методичних вказівок для лабораторної роботи №6.

Експериментальна частина

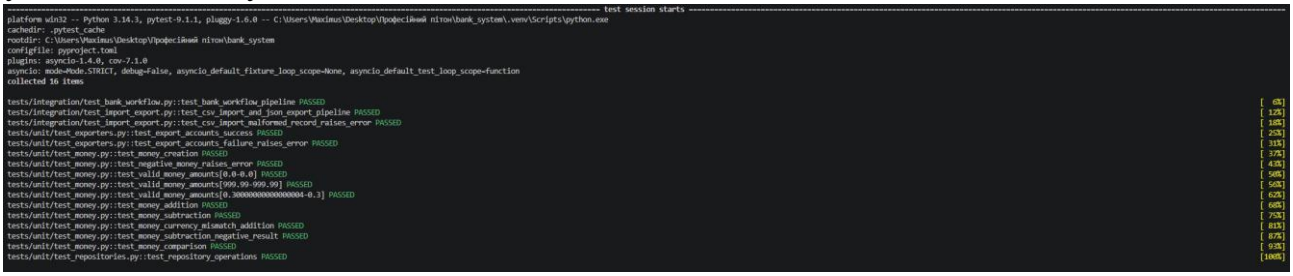
У межах експериментальної перевірки тестового набору було проведено запуск усіх створених модульних (unit) та інтеграційних (integration) тестів із одночасним аналізом покриття коду (code coverage) за допомогою плагіна pytest-cov.

Для запуску тестів у терміналі віртуального середовища проєкту було виконано єдину стандартизовану команду з перевіркою гілок виконання (branch coverage) та виведенням детального звіту:

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> pytest -v --cov=bank_system --cov-branch --cov-report=term-missing --cov-report=html
```

Результати виконання тестів

Під час запуску фреймворк pytest успішно виявив та виконав **16 тестів** (ураховуючи модульні перевірки об'єктів Money, репозиторіїв, експортерів та інтеграційні пайплайни CSV-імпорту й JSON-експорту). Усі тести завершилися успішно зі статусом **PASSED**:



```
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Maximus\Desktop\Професійний пітон\bank_system\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Maximus\Desktop\Професійний пітон\bank_system
configfile: pyproject.toml
plugins: asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 16 items

tests/integration/test_bank_workflow.py::test_bank_workflow_pipeline PASSED                                                
```

Вимога лабораторної роботи / Індивідуального завдання	Стан виконання	Деталі реалізації у проєкті
		інтеграційні (integration/) тести.
Використання assert та pytest.approx	Виконано	Застосовано для перевірки логічних умов та безпечного порівняння чисел із плаваючою комою.
Перевірка винятків через pytest.raises	Виконано	Протестовано генерацію очікуваних бізнес-помилок та винятків валідації (ValueError, RecordValidationError, DataExportError).
Створення фікстур та conftest.py	Виконано	Винесено спільні фабрики об'єктів та тестові дані у файл tests/conftest.py для багаторазового використання.
Параметризовані тести (@pytest.mark.parametrize)	Виконано	Реалізовано для перевірки безлічі вхідних значень і граничних умов об'єкта Money без дублювання коду.
Ізоляція файлової системи через tmp_path	Виконано	Використано вбудовану фікстуру для безпечного тестування CSV-імпорту та JSON-експорту без зміни реальних файлів.
Підміна середовища через monkeypatch	Виконано	Застосовано для тимчасової та безпечної зміни конфігураційних

Вимога лабораторної роботи / Індивідуального завдання	Стан виконання	Деталі реалізації у проєкті
		параметрів та змінних середовища в тестах.
Використання Mock / AsyncMock	Виконано	Протестовано взаємодію з репозиторіями та сервісами, перевірено виклики методів і сценарії збоїв через side_effect.
Інтеграційні тести (Integration Tests)	Виконано	Створено тести повного пайплайну: потоковий імпорт рахунків із CSV → валідація → експорт у JSON.
Аналіз покриття (pytest-cov, branch coverage)	Виконано	Запущено перевірку statement і branch coverage за допомогою pytest-cov, сформовано детальний HTML-звіт.

Контрольні запитання та відповіді

1. **Що таке automated test?**

Це програмний код, який автоматично виконує перевірку правильності поведінки іншого коду без участі людини.

2. **Які основні переваги автоматизованого тестування?**

Швидке знаходження дефектів, запобігання регресіям, безпечний рефакторинг, документування очікуваної поведінки та перевірка edge cases.

3. **Що таке regression?**

Поява нових помилок або порушення працездатності раніше робочого функціоналу внаслідок внесення змін до коду.

4. **Що таке unit test?**

Модульний тест, який перевіряє невелику одиницю поведінки (функцію чи клас) ізольовано від решти системи.

5. **Що таке integration test?**

Тест, який перевіряє коректність взаємодії кількох компонентів у єдиному пайплайні.

6. **Чим integration test відрізняється від unit test?**

Unit test фокусується на одному ізольованому компоненті з використанням заглушок, тоді як integration test охоплює спільну роботу кількох модулів (наприклад, роботу з файловою системою або базами даних).

7. **Що показує test pyramid?**

Ієрархічне співвідношення різних типів тестів: найбільшу частину мають становити дешеві й швидкі unit-тести, меншу — інтеграційні, і найменшу — end-to-end тести.

8. **Чому unit tests зазвичай більше?**

Вони швидкі у виконанні, дешеві в написанні та дозволяють точно локалізувати помилки в конкретному невеликому модулі.

9. **Що таке pytest?**

Популярний, гнучкий і потужний фреймворк для написання та запуску автоматизованих тестів на мові Python.

10. **Як pytest знаходить tests?**

Фреймворк автоматично сканує проєкт у пошуках файлів із назвами test_*.py або *_test.py та функцій, що починаються з test_.

11. **Якою командою запускається pytest?**

Командою pytest у корені робочого каталогу проєкту.

12. **Для чого pytest -v?**

Для детального (verbose) виведення списку всіх тестів із їхніми індивідуальними статусами виконання.

13. **Для чого pytest -x?**

Для негайної зупинки тестування відразу після першого ж невдалиго (failed) тесту.

14. **Що означає Arrange–Act–Assert?**

Стандартний шаблон проєктування тесту: підготовка даних (Arrange), виконання дії (Act) та перевірка результату (Assert).

15. **Для чого використовується assert?**

Для перевірки очікуваних умов; якщо умова хибна, Python генерує виняток AssertionError.

16. **Що таке assertion failure?**

Помилка, яка виникає, коли умова в assert виявляється хибною (фактичний результат не збігається з очікуваним).

17. **Чому floats не завжди слід порівнювати через ==?**

Через особливості двійкового представлення чисел із плаваючою комою, що призводить до невеликих похибок округлення.

18. **Для чого pytest.approx?**

Для безпечного порівняння чисел із плаваючою комою з урахуванням допустимої похибки.

19. **Для чого pytest.raises?**

Для перевірки того, чи викликає фрагмент коду очікуваний тип винятку.

20. **Як перевірити exception message?**

За допомогою параметра match усередині pytest.raises(ErrorType, match="текст").

21. **Як отримати exception object через exc_info?**

Використовуючи контекстний менеджер у вигляді with pytest.raises(ErrorType) as exc_info:, після чого перевіряти exc_info.value.

22. **Що таке boundary testing?**

Метод тестування на граничних значеннях дозволених діапазонів вхідних даних (мінімум, максимум, біля меж).

23. **Які boundaries потрібно тестувати для діапазону 0–100?**

Варто перевіряти значення: -0.01, 0, 0.01, 50, 99.99, 100, 100.01.

24. **Що таке parametrization?**

Підхід, що дозволяє запускати одну й ту саму тестову функцію з різними наборами вхідних аргументів.

25. **Для чого pytest.mark.parametrize?**

Декоратор для передачі різних наборів даних у параметризований тест.

26. **Що дають ids параметризованих tests?**

Задають зрозумілі імена або ідентифікатори для кожної ітерації параметризованого тесту у виводі термінала.

27. **Що таке fixture?**

Функція підготовки тестового середовища, даних або ресурсів, що повторно використовується в тестах.

28. **Для чого використовуються fixtures?**

Для уникнення дублювання коду під час налаштування початкового стану тестів.

29. **Що означає fixture setup?**

Процес підготовки ресурсів та створення об'єктів перед запуском тесту.

30. **Що означає fixture teardown?**

Процес очищення, закриття з'єднань або видалення тимчасових файлів після завершення тесту.

31. **Для чого yield у fixture?**

Для розділення setup (код до yield) та teardown (код після yield) усередині фікстури.

32. **Які fixture scopes підтримує pytest?**

function, class, module, package, session.

33. **Який scope є default?**

function (виконується для кожного тесту окремо).

34. **Для чого conftest.py?**

Для збереження спільних фікстур та налаштувань, які автоматично доступні для всіх тестів у проєкті.

35. **Чому fixtures із conftest.py не імпортують вручну?**

Тому що pytest автоматично виявляє та підключає їх за назвою аргументу в тестовій функції завдяки механізму ін'єкції залежностей.

36. **Що таке tmp_path?**

Вбудована фікстура pytest, яка створює унікальну тимчасову директорію (pathlib.Path) для кожного тесту.

37. **Чому tmp_path корисний?**

Забезпечує ізоляцію файлових операцій, захищаючи реальні дані й усуваючи конфлікти між запусків тестів.

38. **Що таке test isolation?**

Принцип незалежності тестів один від одного, за якого результат одного тесту не впливає на інші.

39. **Для чого monkeypatch?**

Для безпечної та тимчасової підміни змінних середовища, атрибутів, словників чи функцій під час виконання тесту.

40.**Як monkeypatch змінює environment variable?**

За допомогою методу monkeypatch.setenv("NAME", "value").

41.**Як видалити environment variable у test?**

Використовуючи monkeypatch.delenv("NAME", raising=False).

42.**Як підмінити function через monkeypatch?**

За допомогою методу monkeypatch.setattr(module, "func_name", replacement_func).

43.**Що таке test double?**

Загальна назва для будь-яких об'єктів-замінників, що імітують поведінку реальних компонентів у тестах (Dummy, Stub, Fake, Mock, Spy).

44.**Що таке Dummy?**

Об'єкт-заглушка, який передається для заповнення параметрів, але ніколи не використовується всередині тесту.

45.**Що таке Stub?**

Об'єкт, що надає заздалегідь заготовлені відповіді на виклики під час тестування.

46.**Що таке Fake?**

Робоча, але спрощена реалізація компонента (наприклад, InMemoryRepository замість реальної БД).

47.**Що таке Mock?**

Об'єкт, налаштований для перевірки очікуваних взаємодій (чи викликався певний метод із правильними аргументами).

48.**Що таке Spy?**

Справжній об'єкт або заглушка, яка додатково записує інформацію про те, як до неї зверталися.

49. **Для чого Mock?**

Для ізоляції тестованого коду від зовнішніх систем та перевірки фактів взаємодії між об'єктами.

50. **Що таке spec?**

Параметр мок-об'єкта, який обмежує його інтерфейс методами та атрибутами реального класу, запобігаючи викликам неіснуючих методів.

51. **Для чого return_value?**

Для налаштування значення, яке повинен повернути мок-об'єкт при виклику його методу.

52. **Для чого side_effect?**

Для генерації винятків або послідовного повернення різних значень при багаторазових викликах мока.

53. **Як перевірити, що method викликано один раз?**

За допомогою перевірки mock_method.assert_called_once().

54. **Як перевірити parameters виклику?**

Використовуючи mock_method.assert_called_once_with(...) або аналогічні методи перевірки аргументів.

55. **Як перевірити, що method не викликався?**

За допомогою mock_method.assert_not_called().

56. **Що таке state testing?**

Перевірка результату тесту шляхом аналізу кінцевого стану об'єктів (state verification).

57. **Що таке interaction testing?**

Перевірка того, чи викликали компоненти один одного з правильними параметрами (behavior verification).

58. **Коли interaction testing виправдане?**

Коли метод має побічні ефекти, які не можна перевірити через звичайний стан (наприклад, надсилання email або виклик зовнішнього платіжного шлюзу).

59. **Що таке patch?**

Декоратор або контекстний менеджер із бібліотеки unittest.mock для тимчасової підміни об'єктів.

60. **Чому patch потрібно виконувати там, де name використовується?**

Тому що patch замінює посилання на об'єкт саме в тому модулі, де воно імпортується та використовується, а не там, де воно було первинно визначене.

61. **Що таке AsyncMock?**

Спеціальний мок-об'єкт для імітації та перевірки асинхронних корутин (async/await).

62. **Як перевірити awaited call?**

За допомогою методу async_mock.assert_awaited_once_with(...).

63. **Що таке flaky test?**

Нестабільний тест, який може випадково проходити або падати при однакових вхідних даних через залежність від зовнішніх факторів (час, мережа, випадковість).

64. **Чому flaky tests небезпечні?**

Вони підривають довіру до результатів тестування та часто приховують реальні проблеми або марно блокують CI/CD.

65. **Що таке deterministic test?**

Тест, який за однакових умов завжди дає абсолютно однаковий результат.

66. **Чому unit test не повинен залежати від Internet?**

Щоб залишатися швидким, незалежним від мережевих збоїв та повністю детермінованим.

67. **Коли в integration test доцільний real filesystem?**

Коли тестуються реальні операції читання/запису конфігураційних файлів чи експорту з використанням тимчасових папок (tmp_path).

68. **Чому не потрібно mock-ати все?**

Бо надмірне мокування перетворює тест на перевірку самої реалізації моків, втрачаючи сенс перевірки реальної поведінки коду (over-mocking).

69. **Що таке over-mocking?**

Ситуація, коли в тестах підміняється майже все навколо, через що тестується не бізнес-логіка, а самі штучні заглушки.

70. **Що таке coverage?**

Метрика, яка визначає відсоток коду програми, що виконується під час проходження тестів.

71. **Що таке statement coverage?**

Відсоток виконаних інструкцій (рядків коду) відносно їх загальної кількості.

72. **Що таке branch coverage?**

Відсоток пройдених гілок розгалужень (наприклад, обидві гілки умов if/else).

73. **Чим вони відрізняються?**

Statement coverage перевіряє виконання рядків, тоді як branch coverage гарантує, що обидва стани умов (True та False) були перевірені.

74. **Як отримати coverage через pytest-cov?**

За допомогою запуску команди з аргументом --cov=package_name.

75. **Для чого --cov-report=term-missing?**

Для виведення у термінал підсумкової таблиці покриття із зазначенням конкретних номерів не покритих тестами рядків.

76. **Для чого HTML coverage report?**

Для створення інтерактивного візуального звіту в папці htmlcov/, де можна детально переглянути покриття кожного файлу рядок за рядком.

77. **Чи гарантує 100% coverage відсутність bugs?**

Ні, покриття свідчить лише про виконання коду, але не гарантує якості та правильності перевірок (assertions).

78. **Чому meaningful assertions важливіші за coverage percentage?**

Оскільки високий відсоток можна отримати без перевірки реальних результатів (слабкі тести), тоді як осмислені assert гарантують правильність бізнес-логіки.

79. **Що таке coverage threshold?**

Мінімально допустимий відсоток покриття коду, при зниженні якого збірка або тести вважаються неуспішними (fail_under).

80. **Чому tests повинні бути незалежними один від одного?**

Щоб їх можна було запускати в будь-якому порядку або паралельно без ризику впливу спільного стану на результати.

Висновок: Під час виконання лабораторної роботи №6 було освоєно автоматизоване тестування Python-програм за допомогою фреймворку pytest.
Було створено набір модульних та інтеграційних тестів для системи банківських рахунків. Реалізовано фікстури, параметризацію, перевірку винятків, а також використання Mock, tmp_path і monkeypatch для тестування різних сценаріїв.
За допомогою pytest-cov перевірено покриття коду тестами. Усі 16 тестів успішно пройшли перевірку, що підтвердило правильність роботи програми та допомогло зменшити ризик появи помилок під час подальших змін.